Pilot Study

Test Amplification refers exploiting the knowledge of existing test cases, in which developers embed meaningful input data and expected properties in the form of oracles, in order to enhance these manually written tests with respect to an engineering goal (e.g., improve coverage of changes or increase the accuracy of fault localization).

In order to understand how developers amplify test cases in open source software projects, we are conducting the this pilot study. Please answer all the following questions to complete the study. Thanks in advance.

* Indicates required question

1. To what extent do you think the developer may have used/get inspired from the first test case to write the later/following test case? Please indicate your agreement where - (5=Strong Agreement)
(1=Strong Disagreement)

```
public void testForceDeleteAFile2() throws Exception {
    File destination = new File(getTestDirectory(), "copy2.txt");
    destination.createNewFile();
    assertTrue("Copy2.txt doesn't exist to delete", destination.exists());
    FileUtils.forceDelete(destination);
    assertTrue("Check No Exist", !destination.exists());
}
```

```
public void testForceDeleteAFile3() throws Exception {
    File destination = new File(getTestDirectory(), "no_such_file");
    assertTrue("Check No Exist", !destination.exists());
    try {
        FileUtils.forceDelete(destination);
        fail("Should generate FileNotFoundException");
    } catch (FileNotFoundException ignored) {
    }
}
```

Mark only one oval.

1	2	3	4	5
◯	◯	◯	◯	◯

2. Which of the following attributes influenced your decision? Select all that apply -

Check all that apply.

- [ ] Comments (if any)
- [ ] Testcase name
- [ ] Variable naming style
- [ ] Variable or object construction
- [ ] Test data (such as String - "Check No Exist","copy2.txt", Int, Double etc. used)
- [ ] The list of source code methods being invoked
- [ ] The sequence of source code methods being invoked
- [ ] Assertions (i.e. the attributes/methods being invoked on the asserted object such as "destination.exists")
- [ ] How much similar the methods look like
- [ ] Other: _____

Example 2

3. Do you think the developer may have used the first test case to write the later/following test case? Please indicate your agreement where -
(5=Strong Agreement)
(1=Strong Disagreement)

```
/**
 * Test for {@link IOUtils#toInputStream(String)} and {@link IOUtils#toInputStream(String, Str
 * Note, this test utilizes on {@link IOUtils#toByteArray(java.io.InputStream)} and so relies 
 * {@link #testInputStreamToByteArray()} to ensure this method functions correctly.
 *
 * @throws Exception on error
 */
public void testStringToInputStream() throws Exception {
    String str = "Abc123Xyz!";
    InputStream inStream = IOUtils.toInputStream(str);
    byte[] bytes = IOUtils.toByteArray(inStream);
    assertEqualContent(str.getBytes(), bytes);
    inStream = IOUtils.toInputStream(str, null);
    bytes = IOUtils.toByteArray(inStream);
    assertEqualContent(str.getBytes(), bytes);
    inStream = IOUtils.toInputStream(str, "UTF-8");
    bytes = IOUtils.toByteArray(inStream);
    assertEqualContent(str.getBytes("UTF-8"), bytes);
}
```

```
/**
 * Test for {@link IOUtils#toInputStream(CharSequence)} and {@link IOUtils#toInputStream(CharSe
 * Note, this test utilizes on {@link IOUtils#toByteArray(java.io.InputStream)} and so relies 
 * {@link #testInputStreamToByteArray()} to ensure this method functions correctly.
 *
 * @throws Exception on error
 */
public void testCharSequenceToInputStream() throws Exception {
    CharSequence csq = new StringBuilder(""Abc123Xyz!"");
    InputStream inStream = IOUtils.toInputStream(csq);
    byte[] bytes = IOUtils.toByteArray(inStream);
    assertEqualContent(csq.toString().getBytes(), bytes);
    inStream = IOUtils.toInputStream(csq, null);
    bytes = IOUtils.toByteArray(inStream);
    assertEqualContent(csq.toString().getBytes(), bytes);
    inStream = IOUtils.toInputStream(csq, ""UTF-8"");
    bytes = IOUtils.toByteArray(inStream);
    assertEqualContent(csq.toString().getBytes(""UTF-8""), bytes);
}
```

Mark only one oval.

1	2	3	4	5
◯	◯	◯	◯	◯

4. Which of the following attributes influenced your decision? Select all that apply -

Check all that apply.

- [ ] Comments (if any)
- [ ] Testcase name
- [ ] Variable naming style
- [ ] Variable or object construction
- [ ] Test data (such as Strings - "Abc123Xyz!","UTF-8", Int, Double etc. used)
- [ ] The list of source code methods being invoked
- [ ] The sequence of source code methods being invoked
- [ ] Assertions (i.e. the attributes/methods being invoked on the asserted object such as "destination.exists")
- [ ] How much similar the methods look like
- [ ] Other: _____

Example 3

5. Do you think the developer may have used the first test case to write the later/following test case? Please indicate your agreement where -
(5=Strong Agreement)
(1=Strong Disagreement)

```
//-----------------------------------------------------------------------
public void testDeleteQuietlyDir() throws IOException {
    File testDirectory = new File(getTestDirectory(), "testDeleteQuietlyDir");
    File testFile = new File(testDirectory, "testDeleteQuietlyFile");
    testDirectory.mkdirs();
    createFile(testFile, 0);
    assertTrue(testDirectory.exists());
    assertTrue(testFile.exists());
    FileUtils.deleteQuietly(testDirectory);
    assertFalse("Check No Exist", testDirectory.exists());
    assertFalse("Check No Exist", testFile.exists());
}
//-----------------------------------------------------------------------
public void testMoveFileToDirectory() throws Exception {
    File destDir = new File(getTestDirectory(), "moveFileDestDir");
    File movedFile = new File(destDir, testFile1.getName());
    assertFalse("Check Exist before", destDir.exists());
    assertFalse("Check Exist before", movedFile.exists());
    FileUtils.moveFileToDirectory(testFile1, destDir, true);
    assertTrue("Check Exist after", movedFile.exists());
    assertTrue("Original deleted", !testFile1.exists());
}
```

Mark only one oval.

1	2	3	4	5
◯	◯	◯	◯	◯

6. Which of the following attributes influenced your decision? Select all that apply -

Check all that apply.

- [ ] Comments (if any)
- [ ] Testcase name
- [ ] Variable naming style
- [ ] Variable or object construction
- [ ] Test data (such as String - "Check No Exist","testDeleteQuietlyDir", Int, Double etc. used)
- [ ] The list of source code methods being invoked
- [ ] The sequence of source code methods being invoked
- [ ] Assertions (i.e. the attributes/methods being invoked on the asserted object such as "destination.exists")
- [ ] How much similar the methods look like
- [ ] Other: _____

Example 4

7. Do you think the developer may have used the first test case to write the later/following test case? Please indicate your agreement where -
(5=Strong Agreement)
(1=Strong Disagreement)

```java
public void testEqualsNormalizedOnSystem() {
    assertEquals(true, FilenameUtils.equalsNormalizedOnSystem(null, null));
    assertEquals(false, FilenameUtils.equalsNormalizedOnSystem(null, ""));
    assertEquals(false, FilenameUtils.equalsNormalizedOnSystem("", null));
    assertEquals(true, FilenameUtils.equalsNormalizedOnSystem("", ""));
    assertEquals(true, FilenameUtils.equalsNormalizedOnSystem("file.txt", "file.txt"));
    assertEquals(WINDOWS, FilenameUtils.equalsNormalizedOnSystem("file.txt", "FILE.TXT"));
    assertEquals(true, FilenameUtils.equalsNormalizedOnSystem("a\\b\\file.txt", "a/b/file.txt"));
    assertEquals(false, FilenameUtils.equalsNormalizedOnSystem("a/b/", "a/b"));
}
/**
 * Test for https://issues.apache.org/jira/browse/IO-128
 */
public void testEqualsNormalizedError_IO_128() {
    try {
        FilenameUtils.equalsNormalizedOnSystem("//file.txt", "file.txt");
        fail("Invalid normalized first file");
    } catch (IllegalArgumentException e) {
        // expected result
    }
    try {
        FilenameUtils.equalsNormalizedOnSystem("file.txt", "//file.txt");
        fail("Invalid normalized second file");
    } catch (IllegalArgumentException e) {
        // expected result
    }
    try {
        FilenameUtils.equalsNormalizedOnSystem("//file.txt", "//file.txt");
        fail("Invalid normalized both filse");
    } catch (IllegalArgumentException e) {
        // expected result
    }
}
```

Mark only one oval.

1	2	3	4	5
◯	◯	◯	◯	◯

8. Which of the following attributes influenced your decision? Select all that apply -

Check all that apply.

- [ ] Comments (if any)
- [ ] Testcase name
- [ ] Variable naming style
- [ ] Variable or object construction
- [ ] Test data (such as String - "file.txt", Int, Double etc. used)
- [ ] The list of source code methods being invoked
- [ ] The sequence of source code methods being invoked
- [ ] Assertions (i.e. the attributes/methods being invoked on the asserted object such as "destination.exists")
- [ ] How much similar the methods look like
- [ ] Other: _____

Example 5

9. Do you think the developer may have used the first test case to write the later/following test case? Please indicate your agreement where -
(5=Strong Agreement)
(1=Strong Disagreement)

```
//-----------------------------------------------------------------------
public void test_checkEquals_functionality() throws Exception {
    assertEquals(false, IOCase.SENSITIVE.checkEquals("ABC", ""));
    assertEquals(false, IOCase.SENSITIVE.checkEquals("ABC", "A"));
    assertEquals(false, IOCase.SENSITIVE.checkEquals("ABC", "AB"));
    assertEquals(true,  IOCase.SENSITIVE.checkEquals("ABC", "ABC"));
    assertEquals(false, IOCase.SENSITIVE.checkEquals("ABC", "BC"));
    assertEquals(false, IOCase.SENSITIVE.checkEquals("ABC", "C"));
    assertEquals(false, IOCase.SENSITIVE.checkEquals("ABC", "ABCD"));
    assertEquals(false, IOCase.SENSITIVE.checkEquals("", "ABC"));
    assertEquals(true,  IOCase.SENSITIVE.checkEquals("", ""));
    try {
        IOCase.SENSITIVE.checkEquals("ABC", null);
        fail();
    } catch (NullPointerException ex) {}
    try {
        IOCase.SENSITIVE.checkEquals(null, "ABC");
        fail();
    } catch (NullPointerException ex) {}
    try {
        IOCase.SENSITIVE.checkEquals(null, null);
        fail();
    } catch (NullPointerException ex) {}
}
//-----------------------------------------------------------------------
public void test_checkCompare_functionality() throws Exception {
    assertTrue(IOCase.SENSITIVE.checkCompareTo("ABC", "") > 0);
    assertTrue(IOCase.SENSITIVE.checkCompareTo("", "ABC") < 0);
    assertTrue(IOCase.SENSITIVE.checkCompareTo("ABC", "DEF") < 0);
    assertTrue(IOCase.SENSITIVE.checkCompareTo("DEF", "ABC") > 0);
    assertEquals(0, IOCase.SENSITIVE.checkCompareTo("ABC", "ABC"));
    assertEquals(0, IOCase.SENSITIVE.checkCompareTo("", ""));
    try {
        IOCase.SENSITIVE.checkCompareTo("ABC", null);
        fail();
    } catch (NullPointerException ex) {}
    try {
        IOCase.SENSITIVE.checkCompareTo(null, "ABC");
        fail();
    } catch (NullPointerException ex) {}
    try {
        IOCase.SENSITIVE.checkCompareTo(null, null);
        fail();
    } catch (NullPointerException ex) {}
}
```

Mark only one oval.

1	2	3	4	5
◯	◯	◯	◯	◯

10. Which of the following attributes influenced your decision? Select all that apply -

Check all that apply.

- [ ] Comments (if any)
- [ ] Testcase name
- [ ] Variable naming style
- [ ] Variable or object construction
- [ ] Test data (such as String - "ABC", Int, Double etc. used)
- [ ] The list of source code methods being invoked
- [ ] The sequence of source code methods being invoked
- [ ] Assertions (i.e. the attributes/methods being invoked on the asserted object such as "destination.exists")
- [ ] How much similar the methods look like
- [ ] Other: _____

Example 6

11. Do you think the developer may have used the first test case to write the later/following test case? Please indicate your agreement where -
(5=Strong Agreement)
(1=Strong Disagreement)

```java
//-----------------------------------------------------------------------
public void testWrite_charArrayToWriter() throws Exception {
    String str = new String(inData, "US-ASCII");
    ByteArrayOutputStream baout = new ByteArrayOutputStream();
    YellOnFlushAndCloseOutputStream out =
            new YellOnFlushAndCloseOutputStream(baout, true, true);
    Writer writer = new OutputStreamWriter(baout, "US-ASCII");
    IOUtils.write(str.toCharArray(), writer);
    out.off();
    writer.flush();
    assertEquals("Sizes differ", inData.length, baout.size());
    assertTrue("Content differs", Arrays.equals(inData, baout.toByteArray()));
}
//-----------------------------------------------------------------------
public void testWrite_charSequenceToOutputStream_Encoding() throws Exception {
    CharSequence csq = new StringBuilder(new String(inData, "US-ASCII"));
    ByteArrayOutputStream baout = new ByteArrayOutputStream();
    YellOnFlushAndCloseOutputStream out =
            new YellOnFlushAndCloseOutputStream(baout, true, true);
    IOUtils.write(csq, out, "UTF16");
    out.off();
    out.flush();
    byte[] bytes = baout.toByteArray();
    bytes = new String(bytes, "UTF16").getBytes("US-ASCII");
    assertTrue("Content differs", Arrays.equals(inData, bytes));
}
```

Mark only one oval.

1	2	3	4	5
◯	◯	◯	◯	◯

12. Which of the following attributes influenced your decision? Select all that apply -

Check all that apply.

- [ ] Comments (if any)
- [ ] Testcase name
- [ ] Variable naming style
- [ ] Variable or object construction
- [ ] Test data (such as String - "US-ASCII","UTF16", Int, Double etc. used)
- [ ] The list of source code methods being invoked
- [ ] The sequence of source code methods being invoked
- [ ] Assertions (i.e. the attributes/methods being invoked on the asserted object such as "destination.exists")
- [ ] How much similar the methods look like
- [ ] Other: _____

Demographic Information

13. What is you current post * Dropdown

Mark only one oval.

- ( ) PhD Student
- ( ) Masters Student
- ( ) Undergraduate Student
- ( ) Software Developer
- ( ) Others

14. How many years of programming experience you have? Dropdown

Mark only one oval.

- 0-1 year
- 1-3 years
- 3-5 years
- more than 5 years

15. Do you have experience in writing unit test cases?

Mark only one oval.

- Yes
- No
- Maybe

This content is neither created nor endorsed by Google.

Google Forms